

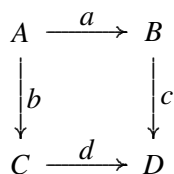
## The DCpic package

This package makes use of `Pictex` (rather, its more modern variant `pictexwd`) to produce commutative diagrams. The file `dcpic.sty` and a manual for DCpic can be found on CTAN or at <http://hilbert.mat.uc.pt/~pedro/DCpic/man/>.

The package provides 11 different types of arrows, and allows you to curve arrows. Arrows don't stretch to match their labels, and there is no switch to change the size of all labels. It is not possible to have labels on both sides of an arrow. For this file, I loaded it using `\usepackage{pictexwd,dcpic}`

An object is given as `\obj(1,2)[3]{4}` where 1,2 are the integer coordinates of the object, 3 is an optional label, and 4 is the object itself.

For example:

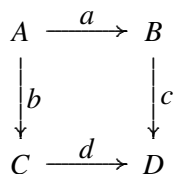


```

\beginDC{\commdiag}[50]
\obj(0,1)[aa]{$A$}
\obj(1,1)[bb]{$B$}
\obj(0,0)[cc]{$C$}
\obj(1,0){$D$}
\mor{aa}{bb}{$a$}
\mor{aa}{cc}{$b$}
\mor{bb}{$D$}{$c$}
\mor{cc}{$D$}{$d$}
\endDC

```

This code puts the object  $A$  at position  $(0,1)$  and labels it `aa` (for internal purposes). It draws an arrow from the object  $A$  to the object  $B$  and puts a label  $a$  in the default position. When an object has no label, the object itself can be used. The “50” is the magnification factor. The same diagram is produced by the following code:



```

\beginDC{\commdiag}[5]
\obj(0,10){$A$}
\obj(10,10){$B$}
...
\mor{cc}{dd}{$d$}
\endDC

```

Specifying a magnification of 5 allows you to position the nodes more finely.

There are the following arrows:

|                            |                               |
|----------------------------|-------------------------------|
| $A \xrightarrow{0} B$      | <code>\solidarrow</code>      |
| $A \xrightarrow{-1} B$     | <code>\dashArrow</code>       |
| $A \xrightarrow{2} B$      | <code>\solidline</code>       |
| $A \xrightarrow{-3} B$     | <code>\dashline</code>        |
| $A \xrightarrow{4} B$      | <code>\dotline</code>         |
| $A \xrightarrow{5} B$      | <code>\injectionarrow</code>  |
| $A \xrightarrow{6} B$      | <code>\aplicationarrow</code> |
| $A \xrightarrow{7} B$      | <code>\surjectivearrow</code> |
| $A \xrightarrow{8} B$      | <code>\equalline</code>       |
| $A \xrightarrow{9} B$      | <code>\doublearrow</code>     |
| $A \xleftrightarrow{10} B$ | <code>10</code>               |

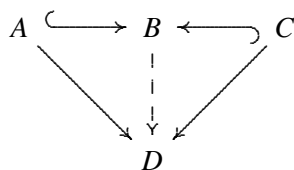
An arrow is invoked by

`\mor{source}{target}{label}[labelposition,type]`

where `labelposition` can be `+1` (`=\atright`) or `-1` (`=\atleft`) and `type` is `0,1,2,...,10` or the corresponding command in the above table. Alternatively, the arrow can be invoked by

`\mor(w,x)(y,z){label}[labelposition,type]`

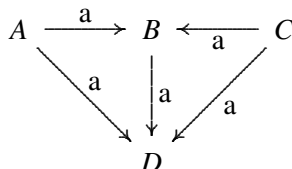
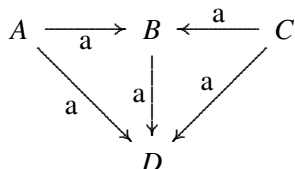
where `(w,x)` and `(y,z)` are the integer coordinates of the source and target. For example, both samples of code below produce the diagram:



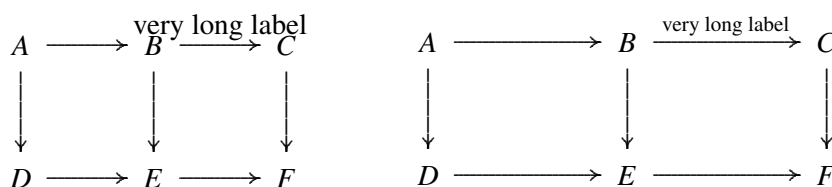
```
\[\begin{cd}\commdiag}[50]
\obj(0,1){A$}
\obj(1,1){B$}
\obj(2,1){C$}
\obj(1,0){D$}
\mor(0,1)(1,1){}\[+1,3]
\mor(2,1)(1,1){}\[+1,3]
\mor(0,1)(1,0){}
\mor(1,1)(1,0){}\[+1,1]
\mor(2,1)(1,0){}
\end{cd}\]
```

```
\[\begin{cd}\commdiag}[50]
\obj(0,1)[a]{A$}
\obj(1,1)[b]{B$}
\obj(2,1)[c]{C$}
\obj(1,0)[d]{D$}
\mor{a}{b}{}\[\atright,\injectionarrow]
\mor{c}{b}{}\[\atright,\injectionarrow]
\mor{a}{d}{}
\mor{b}{d}{}\[\atright,\dashArrow]
\mor{c}{d}{}
\end{cd}\]
```

In the diagram at right below, all arrows have the (default) `\atright` option; in the diagram at left, they have the `\atleft` option.

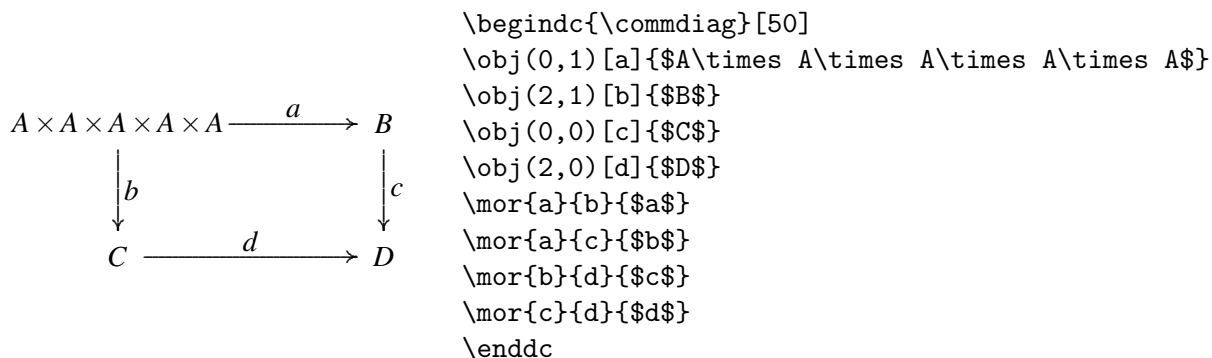


Arrows do not automatically stretch to match long labels:

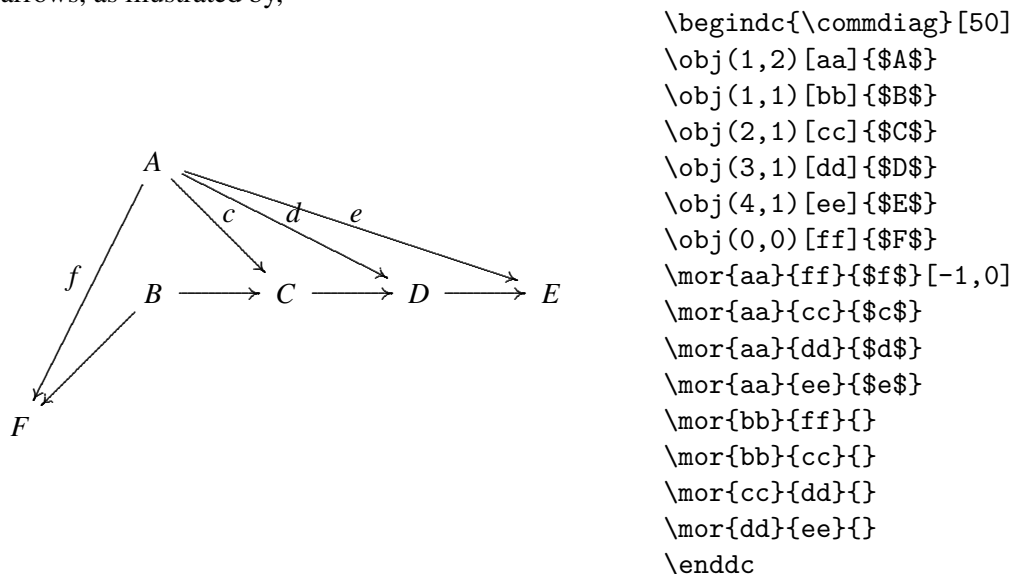


To fix this, replace `\begin{dc}{0}[50]` with `\begin{dc}{0}[5]`, put  $A, B, C \dots$  at  $(0, 10), (15, 10), (30, 10)$  rather than  $(0, 1), (1, 1), (2, 1)$  and shrink the label (`\scriptsize{very long label}`), as in the diagram at right.

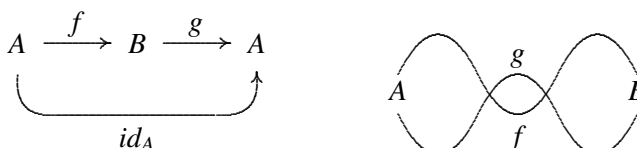
Arrows do now stretch (or contract) to match large objects provided you use labels (rather than coordinates) to determine the start and end of the arrow:

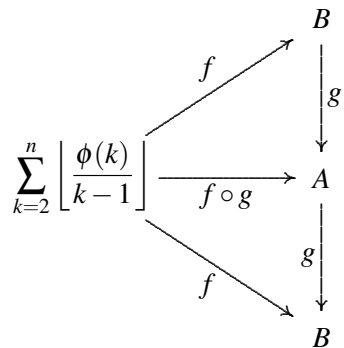


While the syntax of DCpic seems a little clumsy for simple diagrams, it makes it easy to construct complicated diagrams — once the objects are correctly placed, it is easy to add the arrows, as illustrated by,



One of DCpic's strengths is its ability to draw curved arrows. Here are some examples from the "examples" files with the package.





To use DCpic to draw “direct graphs” or “unidirect graph” instead of commutative diagrams, invoke it with `\begin{dc}{digraph}` or `\begin{dc}{unidigraph}`. There is an optional argument `placement` of the object for objects, which (apparently) has no effect for commutative diagrams, and an option argument `correction factors` which, when invoked with numbers other than `[10,10]`, will move the starting or ending point of the arrow.